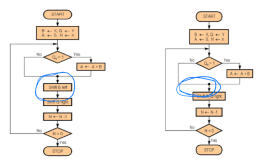


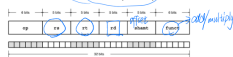
4. multiply (负数就是转换成正数,再2's complement)



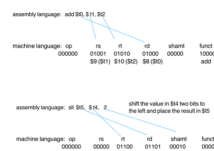
8 1011 Multiplier
Q 1101 Multiplier
1011 A
0000
1011
1011 不断右移

二、(-) R-type Instruction

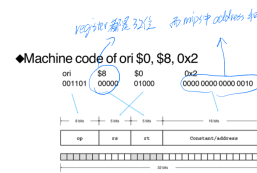
- ◆The machine code
- "Instruction rd, rs, rt" is a line of code in assembly language
- need to translate it to machine language (binary) — assembler
- the machine code has fixed format (32 bits)



ex1: add \$5, \$0, \$2
ex2: slt \$9, \$2
op rs rt rd shamt funct
对5 5 bits $\Rightarrow 2^5 = 32$ register



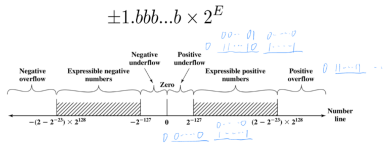
(二) I-type Instruction



两者要逻辑运算或计算时
逻辑运算: 零扩展 ex: ori, andi
计算: 符号扩展 (根据最高位符号) ex: addi, lw, sw
signed 地址偏移量

三、F.T Representation

1 23
biased exponent
biased + E
 2^{n-1} 指数
最大正数: $0.00000010 \dots 0$
最大非零: $0.111111101 \dots 1$ $(2-2^{-23}) \times 2^{128}$



What is the decimal value of the following floating point number represented in binary form (8 bits for biased exp and 23 bits for significant)? [2 pts]
1 1000 1001 111 1100 1101 0000 0000
E = biased exponent - biased = $10001001_2 - (2^{8-1}) = 10$
原式 = $-1.1111001101 \times 2^{10} = -1.1111001101 \times 1024 = -2022.5$

lec 3 (一) 大点中的内容仅能用空扩展

Bit-wise Logic Operation

ori \$8, \$0, 0x2 $\Rightarrow$ load function

异或运算

register 都是 32 bits, 但在这里只有 16 bits

进行零扩展 $\Rightarrow$ 对于非负数, 这个正/负对 ALU 都无所谓, 主要是对于性能很重要

zero extension	
0000 0000 0000 0000 0000 0000 0000 0010	-- zero extended immediate operand
0000 0000 0000 0000 0000 0000 0000 0000	-- data in register \$0
0000 0000 0000 0000 0000 0000 0000 0010	-- result, put in register \$8

2. andi \$8, \$0, 0x2

只有同时为 1 时, 才为 1

3. xori \$8, \$0, 0x2

只有不同时, 才为 1

4. nori \$8, \$0, 0x2

只有同时为 0 时, 才为 1

一、Conditional Set Instruction

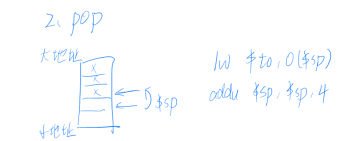
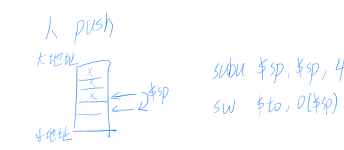
1. set flag (0: false, 1: true)

- slt d, s, t # \$s and \$t contains signed integers in 2's complement
set \$d = 1 if \$s < \$t; else, set \$d = 0
- sltu d, s, t # used with unsigned integers
- slti d, s, imm # Set on Less Than Immediate + signed
if \$s < imm, set \$d = 1; else, set \$d = 0
- sltiu d, s, imm # used with unsigned integers + immediate

Exercise
A. After the following code executes, what value is in register \$1?
addiu \$1, \$0, -25
addiu \$1, \$1, 25
slt \$1, \$1, 0
B. After the following code executes, what value is in register \$1?
addiu \$1, \$0, -25
addiu \$1, \$1, 25
sltiu \$1, \$1, 0
Hint: check the bit patterns in \$5 and \$6

二、Stack and Subroutine in MIPS

(一) Stack



2. Subtraction Instruction

sub/subu d, s, t $\Rightarrow$ s - t ignore overflow
addiu: 针对 register-immediate addiu d, s, constant
因为又是 register-immediate 结合, 所以不会涉及到 extensive 100 个问题
ALU 会根据 constant 的符号, 进行扩展。
EX: -1111 1111 1111 1111 1010 1101 1110 1101

2. Subroutine

The jal and jr Instruction

- ◆jal sub
- (1) set \$ra to PC + 4 (see next slide for the details)
- the result is that \$ra stores the address of the (n+2)th instruction (suppose jal is the nth instruction)
- in memory: branch delay slot 1 instruction that will return to
- (2) set PC to sub
- ◆jr \$ra
- set PC to \$ra

Calling Convention — Use of Registers

- ◆Different roles of registers
- \$t0 - \$t9: subroutines can use without caution
- \$s0 - \$s7: saved registers; subroutines should not use (if need to use it, save the value first and restore the value after use)
- \$a0 - \$a3: contain arguments for the subroutine 用于传参数
- \$v0 - \$v1: contain values returned from the subroutine 用于返回结果

Calling Convention — Use of Registers

◆Example: is the following code a good convention?

```
add $t0, $s0, $a3 # calculate an important sum in $t0
jal $ra           # call a subroutine
nop              # branch delay
mul $a4, $v0, $v1 # multiply the sum
```

NOT good: the main routine used the temporary register \$t0, for which any subroutine can use without caution

3. Multiplication Instruction

1. mult s, t / multu s, t
signed unsigned



2. 当求结果时, 会存储在 2 个寄存器中, 共 32 + 32 = 64 bits
若想将 64 $\rightarrow$ 32 bits, 可以用:
mfhi d 将高位移到 register d 中
mflo d 将低位移到 register d 中

4. Division Instruction

div/divu s, t
mfhi d 将商移到 register d 中
mflo d 将余数移到 register d 中

5. Set Base Address and Memory Address

lui d, const: 将 const (16 bits) 放入 d 的前 2 个 bytes (高 16 位)
ori d, d, imm: 将 immediate 与 d 再进行 按位或运算
register d 32 bits
0 extension

b. memory $\xleftrightarrow{\text{load}}$ register (ALU calculate)
store

lw d, offset(b): 把内存地址为 b+offset 的字节 load 到 d 中
sw d, offset(b): 把 d 中的值, store 到 b+offset 的内存地址中

Stack-based Linkage Convention

- ◆Subroutine Call (done by Caller)
 - Push registers \$t0 - \$t9 to stack (callee may use them)
 - Put argument values into \$a0 - \$a3
- ◆Subroutine Beginning (done by Callee)
 - If this subroutine will call another subroutine, push \$ra to stack
 - Push registers \$s0 - \$s7 to stack (caller may use them)

Stack-based Linkage Convention

- ◆Subroutine Body
 - May use \$t0 and \$t9 if pushed to stack
 - May call another subroutine
- ◆Subroutine Ending (done by the Callee)
 - Put returned values in \$v0 - \$v1
 - Pop registers \$s0 - \$s7 from stack (to restore the value)
 - Pop \$ra
 - Return to caller using jr \$ra

